

INVENTORY TRACKING SYSTEM

System Design and Requirements Document

Overview/Business Problem:

Operations teams often lack a centralized system to track inventory across suppliers and locations. This can often lead to inconsistent stock levels, delayed replenishment, and limited visibility into inventory availability.

This project aims to design and implement an inventory tracking system that improves visibility, ensures accurate inventory status, and supports improved operational decision-making.

Objectives:

- Improve visibility into inventory across suppliers and locations.
- Ensure accurate tracking of inventory levels.
- Enable users to query inventory data efficiently.
- Simulate real-world enterprise inventory challenges.

Stakeholders:

Inventory Manager
Inventory Team
Operations Team
Supplier

System Requirements:

Functional Requirements:

- The system must store inventory items and quantities.
- The system must associate inventory with a supplier.
- The system must track inventory by location.
- The system must allow updates to inventory levels.
- The system must track availability status.
- The system must allow users to query inventory data.
- The system must define and track reorder points for inventory items.
- The system must flag inventory items when stock levels fall below the defined reorder point.

Non-functional Requirements:

- The system should ensure data accuracy and consistency.
- The system should support multiple users querying data.
- The system should simulate cloud-based inventory workflows.
- The system should maintain basic data security.

Process Flow:

Inventory is received from suppliers and recorded in the system. Each inventory item is assigned to a specific location and linked to a supplier. The system updates inventory quantities as items are used, transferred, or replenished. Based on predefined thresholds, the system updates inventory status (e.g., available, low stock, out of stock). Users can query the system to view inventory by supplier, location, or item.

System Architecture:

The system follows a modular, layered architecture separating database access, business logic, and user interaction. This layered approach improves maintainability, scalability, and separation of concerns.

User (CLI Interface)



app.py (Controller Layer)



inventory.py (Business Logic Layer)



db.py (Database Access Layer)



SQLite Database

Component Responsibilities:

- **app.py**: Handles user interaction, menu navigation, and workflow control
- **inventory.py**: Implements business logic, validation, and SQL queries
- **db.py**: Manages database connections
- **utils.py**: Provides reusable input validation functions

Data Flow:

1. User initiates an action (e.g., add product, update quantity)
2. Input is validated using utility functions
3. Request is passed to the business logic layer
4. Business logic executes SQL queries via the database layer
5. Database returns results
6. Results are displayed to the user

Data Model:

The system uses a normalized relational schema with foreign key relationships to maintain data integrity.

Inventory

- product_id (PK)
- product_name
- quantity
- reorder_point
- status (e.g., 'In Stock', 'Low Stock')
- supplier_id (FK)
- location_id (FK)

Supplier

- supplier_id (PK)
- supplier_name

Location

- location_id (PK)
- location_name

Assumptions and Constraints:**Assumptions:**

- Each product is uniquely identified by its name
- Suppliers and locations are predefined or added by users
- Inventory quantities are non-negative integers

Constraints:

- SQLite is used as a lightweight database (not optimized for high concurrency)
- System is designed for small-scale simulation, not enterprise deployment
- CLI interface limits usability compared to graphical interfaces